

Experiment No: 1

Date: _____

TDMA MULTIPLE ACCESS TECHNIQUES

Aim: To Simulate and verify the Operation of TDMA (Time Division Multiple Access) for wireless communication

Tools Required: MATLAB R2015a/ Simulink library

Theory: Multiple access techniques allow users to share a single communication channel among multiple users. TDMA (Time Division Multiple Access) assigns different time slots to each user on the same frequency. All users share the same frequency band but are assigned specific, non-overlapping time slots to transmit data. When the channel is assigned for a specific amount of time, each user adds a bit in the data for synchronization purposes. Sender and receiver devices can synchronize their clocks at regular times with the help of a synchronization bit. The bandwidth of one channel is divided in such a way that it cannot interfere with the signal of another user. The Global System for Mobile Communications (GSM network) uses TDMA method.

Procedure:

- 1) Generate multiple users with data to transmit.
- 2) Allocate time slots to each user.
- 3) Simulate the transmission of data for each user in their respective time slots.
- 4) Analyze the system performance in terms of Bit Error Rate.

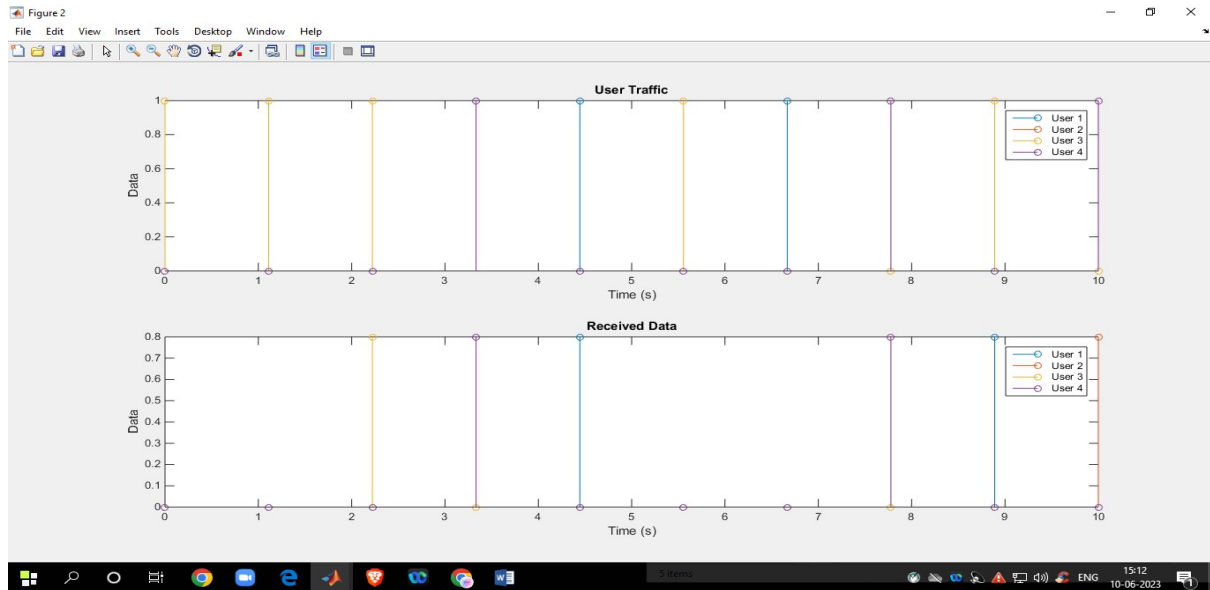
MATLAB Code:

```
TDMA System
% Step 1: Define System Parameters
numUsers = 4;
timeSlotDuration = 1; % seconds
totalTimeSlots = 10;
channelGain = 0.8;
% Step 2: Generate User Traffic
userData = randi([0, 1], numUsers, totalTimeSlots);
```

```
% Step 3: Create Time Slots
timeSlots = linspace(0, timeSlotDuration*totalTimeSlots, totalTimeSlots);
% Step 4: Allocate Time Slots to Users
userSlots = mod(0:totalTimeSlots-1, numUsers) + 1;
% Step 5: Simulate Transmission
receivedData = zeros(numUsers, totalTimeSlots);
for slot = 1:totalTimeSlots
    for user = 1:numUsers
        if userSlots(slot) == user
            % Simulate transmission for the current user in the time slot
            transmittedData = userData(user, slot);
            % Simulate channel effects
            receivedData(user, slot) = transmittedData * channelGain;
        end
    end
end
% Step 6: Evaluate Performance Metrics (e.g., BER)
bitErrorRate = sum(sum(xor(receivedData, userData))) / (numUsers * totalTimeSlots);
% Step 7: Visualize Results
figure;
subplot(2, 1, 1);
stem(timeSlots, userData');
title('User Traffic');
xlabel('Time (s)');
ylabel('Data');
legend('User 1', 'User 2', 'User 3', 'User 4');
subplot(2, 1, 2);
stem(timeSlots, receivedData');
title('Received Data');
xlabel('Time (s)');
ylabel('Data');
legend('User 1', 'User 2', 'User 3', 'User 4');

disp(['Bit Error Rate: ', num2str(bitErrorRate)]);

%Change the parameters and check the result.
Output :
Bit Error Rate: 0.375
```



Observation- Change the parameters and check the results.

Result: Simulation results of FDMA for wireless communication has been verified

Experiment No: 2

Date: _____

FDMA MULTIPLE ACCESS TECHNIQUES

Aim: To Simulate and verify the Operation of FDMA (Frequency Division Multiple Access) for wireless communication

Tools Required: MATLAB R2015a/ Simulink library

Theory: Multiple access techniques allows users to share a single communication channel among multiple users. FDMA (Frequency Division Multiple Access) divides the channel into separate frequency bands for each user. Each user is allocated a specific frequency band for transmitting data. Each user transmits data in the form of an analog signal, and it produces sine waves. Communication takes place only in fixed frequency bands.

Procedure:

For FDMA Simulation:

- 1) Generate multiple users with data to transmit.
- 2) Divide the available bandwidth into frequency bands.
- 3) Allocate frequency bands to each user.
- 4) Simulate the transmission of data for each user in their allocated frequency bands.
- 5) Analyse the system performance in presence of Noise.

MATLAB Code:

FDMA:

```
% System parameters
totalBandwidth = 10e6; % Total available bandwidth (Hz)
numUsers = 5; % Number of users
carrierFrequency = 1e6; % Carrier frequency (Hz)
userBandwidth = totalBandwidth / numUsers; % Bandwidth allocated to each user (Hz)
% Time parameters
samplingFrequency = 100e6; % Sampling frequency (Hz)
timeDuration = 1e-3; % Simulation duration (s)
time = 0:1/samplingFrequency:timeDuration;
% Generate user signals
userSignals = zeros(numUsers, length(time));
for i = 1:numUsers
```

```

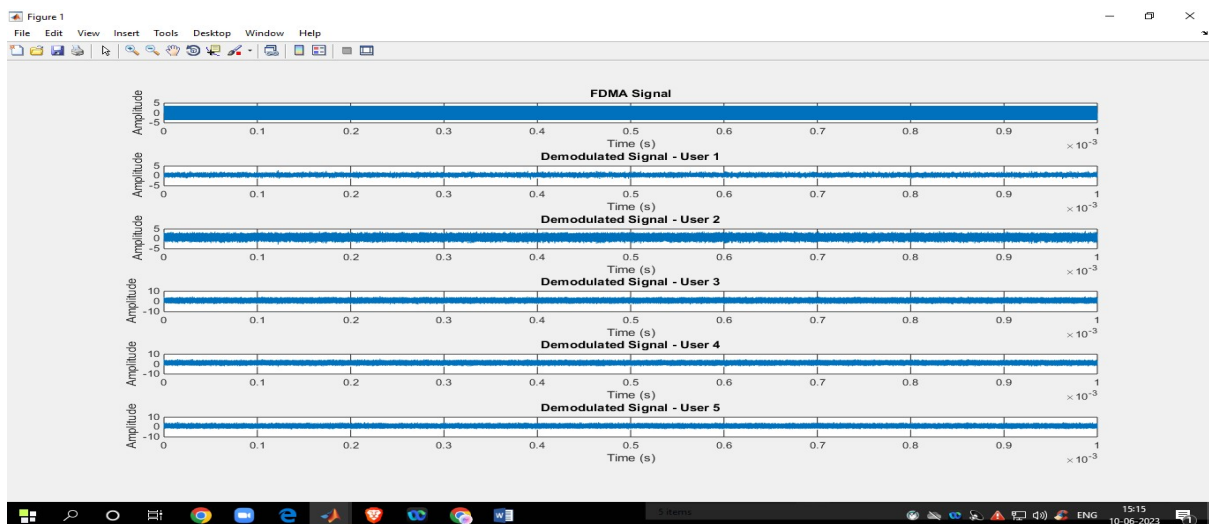
userFrequency = carrierFrequency + (i-1) * userBandwidth; % Frequency of user signal
userSignals(i, :) = sin(2*pi*userFrequency*time);
end

```

```

% Create the FDMA signal
fdmaSignal = sum(userSignals, 1);
% Add noise to the FDMA signal
snr = 10; % Signal-to-Noise Ratio (in dB)
noisySignal = awgn(fdmaSignal, snr, 'measured');
% Perform signal demodulation
demodulatedSignals = zeros(numUsers, length(time));
for i = 1:numUsers
userFrequency = carrierFrequency + (i-1) * userBandwidth; % Frequency of user signal
demodulatedSignals(i, :) = noisySignal .* sin(2*pi*userFrequency*time);
end
% Plot the original user signals and the demodulated signals figure;
subplot(numUsers+1, 1, 1);
plot(time, fdmaSignal);
title('FDMA Signal');
xlabel('Time (s)');
ylabel('Amplitude');
for i = 1:numUsers
subplot(numUsers+1, 1, i+1);
plot(time, demodulatedSignals(i, :));
title(['Demodulated Signal - User ', num2str(i)]);
xlabel('Time (s)');
ylabel('Amplitude');
end

```



Observation- Change the parameters and check the results.

Result: Simulation results of FDMA for wireless communication has been verified

Experiment No: 3

Date: _____

CDMA MULTIPLE ACCESS TECHNIQUES

Aim: To Simulate and verify the Operation of CDMA (Code Division Multiple Access) for wireless communication.

Tools Required: MATLAB R2015a/ Simulink library

Theory: Multiple access techniques allows users to share a single communication channel among multiple users. CDMA (Code Division Multiple Access) uses unique codes to differentiate users sharing the same frequency band at the same time. In CDMA, every communication channel uses the full available spectrum. Individual conversations are encoded with a pseudo-random digital sequence which distinguishes them from one another on the same frequency band. Therefore, multiple users can use the same frequency spectrum simultaneously. Decoding is done by correlating the received signal with the known code of the desired user, effectively extracting that user's information from the combined stream. This technology is widely used in mobile phone services and other wireless communication mediums.

Procedure CDMA Simulation:

- 1) Generate multiple users with data to transmit.
- 2) Assign unique codes to each user.
- 3) Simulate the transmission of data for all users sharing the same frequency band but using different codes.
- 4) Analyze the system performance in terms of BER.

MATLAB Code:

```
CDMA:
% CDMA Simulation Parameters
numBits = 1000; % Number of bits per user
chipRate = 1e6; % Chip rate (chips per second)
snr = 10; % Signal-to-Noise Ratio (dB)
% Generate random data bits for User 1 and User 2
user1Bits = randi([0, 1], 1, numBits);
user2Bits = randi([0, 1], 1, numBits);
% BPSK Modulation
user1Symbols = 2 * user1Bits - 1; % Map 0s to -1 and 1s to 1
user2Symbols = 2 * user2Bits - 1;
% Chip-level Spreading (using a simple chip sequence)
chipSequence = [1, -1, 1, 1, -1, 1, -1, -1]; % Chip sequence for spreading
```

```

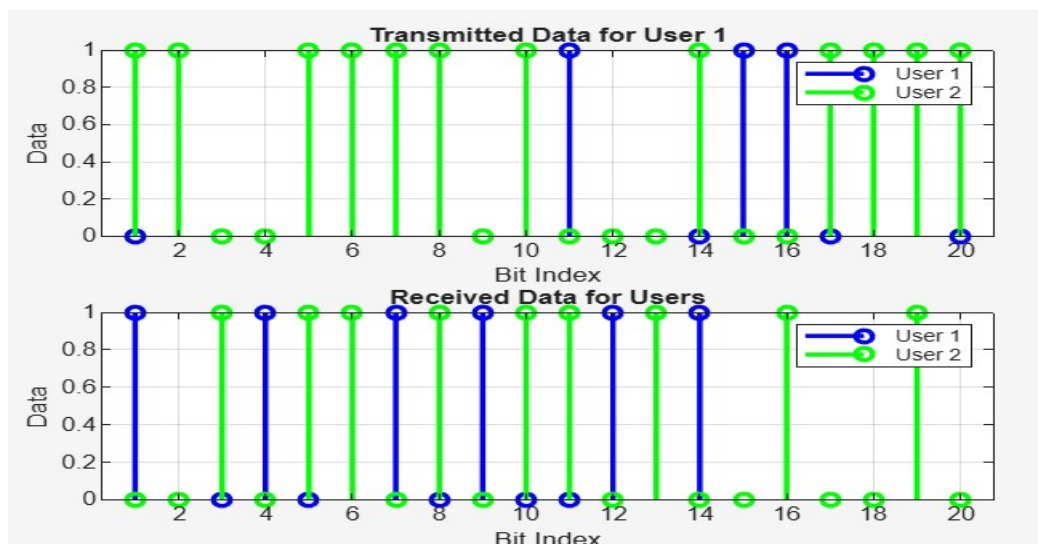
user1SpreadSymbols = kron(user1Symbols, chipSequence);
user2SpreadSymbols = kron(user2Symbols, chipSequence);
% Add AWGN (Additive White Gaussian Noise)
noiseVar = 10^(-snr/10); % Noise variance
user1NoisySymbols = user1SpreadSymbols + sqrt(noiseVar/2) * randn(1,
length(user1SpreadSymbols));
user2NoisySymbols = user2SpreadSymbols + sqrt(noiseVar/2) * randn(1,
length(user2SpreadSymbols));
% Matched Filtering (correlation with chip sequence)
user1FilteredSymbols = filter(fliplr(chipSequence), 1, user1NoisySymbols);
user2FilteredSymbols = filter(fliplr(chipSequence), 1, user2NoisySymbols);
% Symbol Detection (using correlation with chip sequence)
user1DetectedBits = user1FilteredSymbols(1:length(user1Symbols)) > 0;
user2DetectedBits = user2FilteredSymbols(1:length(user2Symbols)) > 0;
% Bit Error Rate (BER) Calculation
berUser1 = sum(user1DetectedBits ~= user1Bits) / numBits;
berUser2 = sum(user2DetectedBits ~= user2Bits) / numBits;
% Display results
disp(['User 1 BER: ', num2str(berUser1)]);
disp(['User 2 BER: ', num2str(berUser2)]);
subplot(2, 1, 1);
stem(user1Bits, 'b', 'LineWidth', 2);
title('Transmitted Data for User 1');
xlabel('Bit Index'); ylabel('Data');
hold on;
stem(user2Bits, 'g', 'LineWidth', 2);
legend('User 1', 'User 2');
grid on;
subplot(2, 1, 2);
stem(user1DetectedBits, 'b', 'LineWidth', 2);
title('Received Data for Users');
xlabel('Bit Index');
ylabel('Data');
hold on;
stem(user2DetectedBits, 'g', 'LineWidth', 2);
legend('User 1', 'User 2');
grid on;

```

Output:

User 1 BER: 0.523

User 2 BER: 0.535



Observation- Change the parameters and check the results.

Result: Simulation results of CDMA for wireless communication has been verified

Experiment No: _

Date: _____

Experiment No: 4

Date: _____

Analyze the Peak Error Rate Performance in Wireless LAN**Aim: To Analyze the Peak Error Rate Performance in Wireless LAN*****Tools Required: MATLAB R2015a/ Simulink library*****Theory:**

A wireless LAN (WLAN), . IEEE 802.11 is a wireless computer network that links two or more devices using wireless communication to form a local area network (LAN) within a limited area such as a home, school, computer laboratory, campus, or office building. This gives users the ability to move around within the area and remain connected to the network.

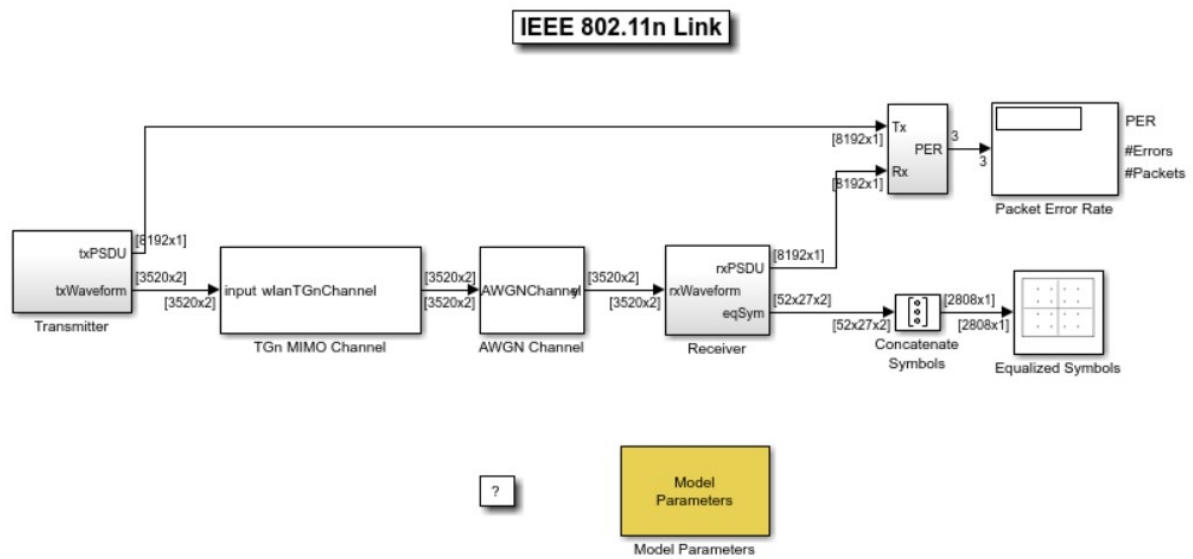
We will simulate an IEEE® 802.11n™ HT (high Throughput) link in Simulink® with WLAN Toolbox. An 802.11n HT link with a fading channel is simulated in this model. Multiple packets are transmitted through a 2-by-2 TGn (IEEE 802.11n Task Group) standard MIMO channel, demodulated and the PSDUs recovered. The PSDUs (Physical Service Data Unit (PSDU) is the data unit being sent down from the MAC layer for transmission on the wireless medium.) are compared to those transmitted to determine the packet error rate. Packet detection, timing synchronization, carrier frequency offset correction and pilot phase tracking are performed by the receiver.

An 802.11n link is modeled in Simulink by using MATLAB Function blocks to call WLAN Toolbox functions.

<https://in.mathworks.com/help/wlan/ug/802-11n-link-in-simulink.html>

The model has four main parts:

- Transmitter: Generates a random PSDU and creates an 802.11n HT packet.
- Channel: Models a TGn 2x2 MIMO channel with AWGN.
- Receiver: Recovers the transmitted PSDU by performing packet detection, time and frequency synchronization, MIMO channel estimation and PSDU demodulation and decoding.
- Analysis: Compares the transmitted and recovered PSDUs to determine the packet error rate, and displays the equalized symbols.



Transmitter: The Transmitter block creates a random PSDU and encodes the bits to create a single packet waveform. The [wlanWaveformGenerator](#) function is called within the Packet Generator block to generate a waveform for a packet. An idle period is added after each packet to create periodic bursts.

Receiver: The receiver has two components: packet detection and packet recovery.

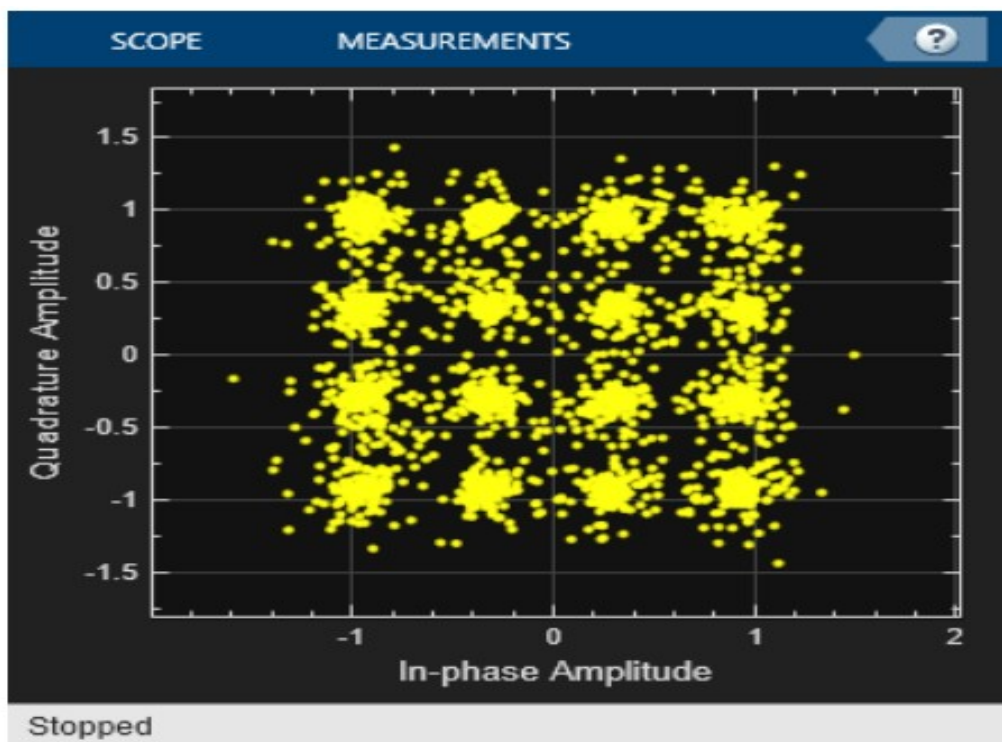
To run the command type in command window

```
openExample('wlan/HTLinkModelWLANExample')
```

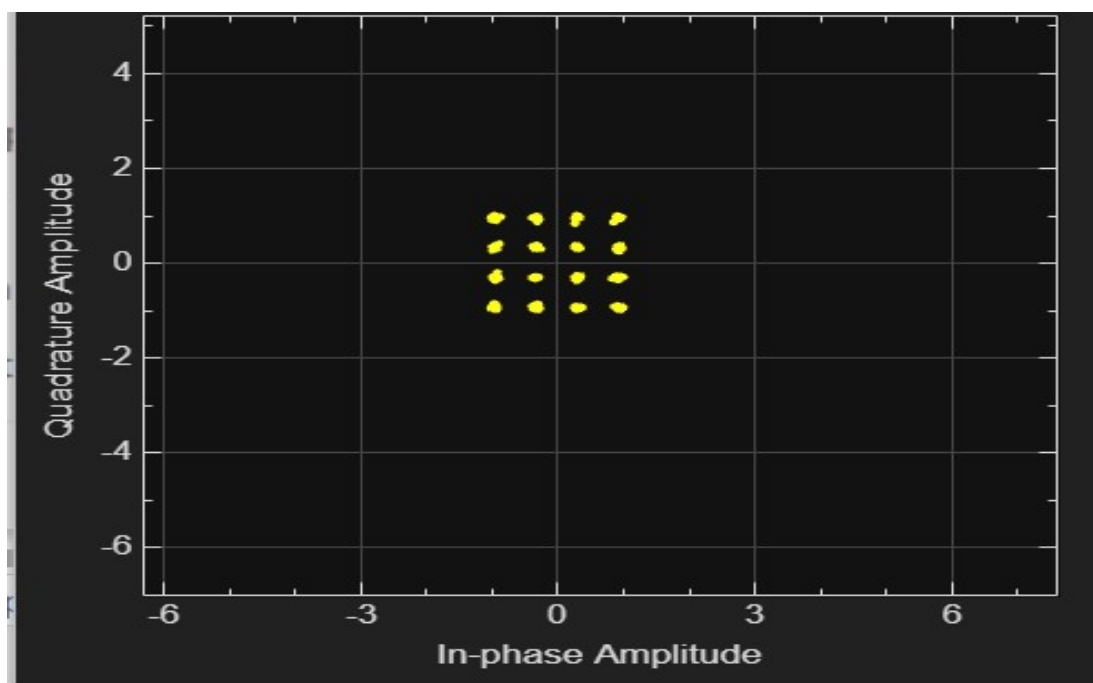
Observations: Change the parameters with modulation and encoding scheme and measure Packet Error Rate.

Results and Displays:

When the simulation is run, the packet error rate is displayed. This is updated after each packet is processed. The equalized data symbols are also displayed for each packet processed. By default, 200 packets are simulated.



Check Results for different values of SNR given in AWGN Channel and Modulation schemes under parameter settings



Further, to check PER execute following command, Change the parameters for different Channel Coding and antennas under Waveform Configuration Section and observe the results

To run the command type in command window

`openExample('wlan/HTMIMOPacketErrorRateExample')`

Waveform Configuration

%% Waveform Configuration

% An 802.11n HT transmission is simulated in this example. The HT format configuration object, `wlanHTConfig`, contains the

% format specific configuration of the transmission. The properties of the object contain the configuration. In this example the object is configured for a 20 MHz channel bandwidth, 2 transmit antennas, 2 space time streams and no space time block coding.

% Create a format configuration object for a 2-by-2 HT transmission

`cfgHT = wlanHTConfig;`

`cfgHT.ChannelBandwidth = 'CBW20';` % 20 MHz channel bandwidth

`cfgHT.NumTransmitAntennas = 1;` % 2 transmit antennas

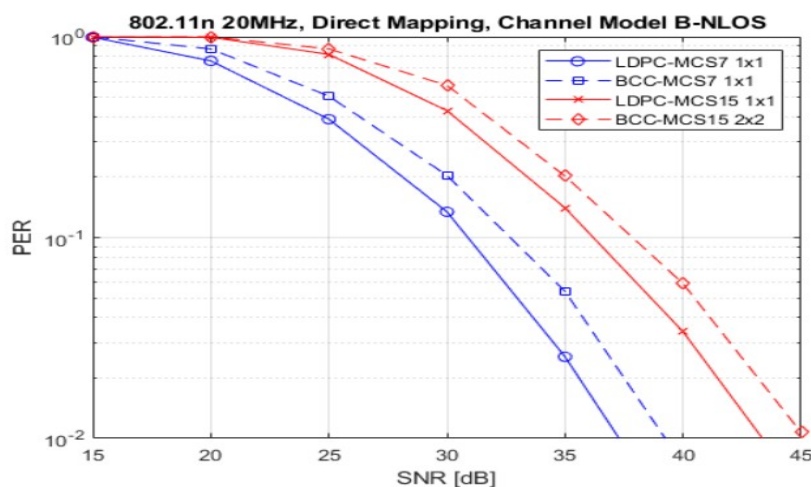
`cfgHT.NumSpaceTimeStreams = 1;` % 2 space-time streams

`cfgHT.PSDULength = 1000;` % PSDU length in bytes

`cfgHT.MCS = 7;` % 2 spatial streams, 64-QAM rate-5/6

`cfgHT.ChannelCoding = 'BCC';` % BCC channel coding

Change the channel coding parameters from BCC to LDPC and observe and compare the results.



Experiment No: 5**Date:** _____**Simulation of Cellular Fundamentals**

Aim: To analyze number of data and control channels in Cellular system based on number of cells, number of users, data channel and control channel.

Tools Required: MATLAB R2015a/ Simulink library

Theory: This fundamental principle is the foundation for all modern wireless communication systems, since it enables a fixed number of channels to serve an arbitrarily large number of subscribers by reusing the channels throughout the coverage region. Based on the available bandwidth system allocates few data and control channels, according to number of users per cell and available cluster size for frequency reuse

```
clc;
clear all;
close all;

num_cells = 4;
num_users_per_cell = 100;
total_users = num_cells * num_users_per_cell;
control_channel_bandwidth = 1e6;
total_bandwidth = 50e6;
data_channel_bandwidth = total_bandwidth - control_channel_bandwidth;
data_channel_bandwidth_pc = 50e3;
control_channel_bandwidth_sys = num_cells * 1e6;
data_channel_bandwidth_sys = num_cells * 50e3; %

num_data_channels = data_channel_bandwidth / data_channel_bandwidth_pc;
num_control_channels = control_channel_bandwidth /
data_channel_bandwidth_pc;
num_data_channels_pc = ceil(num_data_channels / num_cells);
num_control_channels_pc = ceil(num_control_channels / num_cells);

fprintf('Cellular Network Evaluation:\n');
fprintf('Number of Cells: %d\n', num_cells);
fprintf('Number of Users per Cell: %d\n', num_users_per_cell);
fprintf('Total Number of Users: %d\n', total_users);
fprintf('Total Bandwidth: %.2f MHz\n', total_bandwidth / 1e6);
```

```
fprintf('Control Channel Bandwidth: %.2f MHz\n', control_channel_bandwidth / 1e6);
fprintf('Data Channel Bandwidth: %.2f MHz\n', data_channel_bandwidth / 1e6);
fprintf('Number of Data Channels per cell: %d\n', num_data_channels_pc);
fprintf('Number of Control Channels per cell: %d\n', num_control_channels_pc);

num_cells = 7;
num_users_per_cell = 100;
total_users = num_cells * num_users_per_cell;
control_channel_bandwidth = 1e6;
total_bandwidth = 50e6;
data_channel_bandwidth = total_bandwidth - control_channel_bandwidth;
data_channel_bandwidth_pc = 50e3;
control_channel_bandwidth_sys = num_cells * 1e6;
data_channel_bandwidth_sys = num_cells * 50e3;

num_data_channels = data_channel_bandwidth / data_channel_bandwidth_pc;
num_control_channels = control_channel_bandwidth / data_channel_bandwidth_pc;
num_data_channels_pc = ceil(num_data_channels / num_cells);
num_control_channels_pc = ceil(num_control_channels / num_cells);

fprintf('Cellular Network Evaluation:\n');
fprintf('Number of Cells: %d\n', num_cells);
fprintf('Number of Users per Cell: %d\n', num_users_per_cell);
fprintf('Total Number of Users: %d\n', total_users);
fprintf('Total Bandwidth: %.2f MHz\n', total_bandwidth / 1e6);
fprintf('Control Channel Bandwidth: %.2f MHz\n', control_channel_bandwidth / 1e6);
fprintf('Data Channel Bandwidth: %.2f MHz\n', data_channel_bandwidth / 1e6);
fprintf('Number of Data Channels per cell: %d\n', num_data_channels_pc);
fprintf('Number of Control Channels per cell: %d\n', num_control_channels_pc);

num_cells = 12;
num_users_per_cell = 100;
total_users = num_cells * num_users_per_cell;
control_channel_bandwidth = 1e6;
total_bandwidth = 50e6;
data_channel_bandwidth = total_bandwidth - control_channel_bandwidth;
data_channel_bandwidth_pc = 50e3;
control_channel_bandwidth_sys = num_cells * 1e6;
```

```
data_channel_bandwidth_sys = num_cells*50e3; %

num_data_channels = data_channel_bandwidth/data_channel_bandwidth_pc;
num_control_channels=control_channel_bandwidth/
data_channel_bandwidth_pc;
num_data_channels_pc = ceil(num_data_channels/num_cells);
num_control_channels_pc = ceil(num_control_channels/num_cells);

fprintf('Cellular Network Evaluation:\n');
fprintf('Number of Cells: %d\n', num_cells);
fprintf('Number of Users per Cell: %d\n', num_users_per_cell);
fprintf('Total Number of Users: %d\n', total_users);
fprintf('Total Bandwidth: %.2f MHz\n', total_bandwidth / 1e6);
fprintf('Control Channel Bandwidth: %.2f MHz\n', control_channel_bandwidth
/ 1e6);
fprintf('Data Channel Bandwidth: %.2f MHz\n', data_channel_bandwidth / 1e6);
fprintf('Number of Data Channels per cell: %d\n', num_data_channels_pc);
fprintf('Number of Control Channels per cell: %d\n',
num_control_channels_pc);

clc; clear; close all;

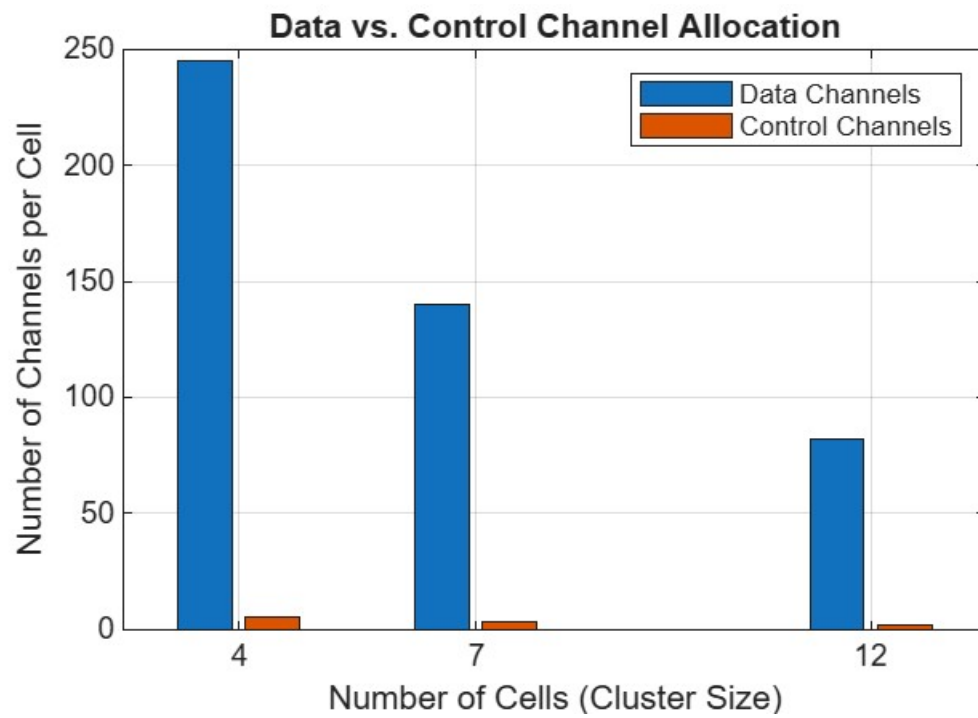
N_cells = [4, 7, 12];
BW_total = 50e6;
BW_control_total = 1e6;
ch_spacing = 50e3;

total_data_ch = (BW_total - BW_control_total) / ch_spacing;
total_ctrl_ch = BW_control_total / ch_spacing;

results = [ceil(total_data_ch ./ N_cells); ceil(total_ctrl_ch ./ N_cells)];

figure('Color', 'w');
hb = bar(N_cells, results, 'grouped');
grid on;

xlabel('Number of Cells (Cluster Size)');
ylabel('Number of Channels per Cell');
title('Data vs. Control Channel Allocation');
legend('Data Channels', 'Control Channels');
```



Experiment No: 6

Date: _____

Modelling and simulation of Multipath fading channel***Aim: To model and simulate a Multipath fading channel using MATLAB.*****Tools Required:** Matlab R2015a/ Simulink library

Theory: In wireless communication, Multipath fading occurs when a radio signal travels from the transmitter to the receiver via multiple paths, rather than a single, direct path. A multipath fading channel model characterizes the random fluctuations in a received wireless signal's amplitude, phase, and angle of arrival, caused by scattering, reflection, and diffraction of radio waves. Multipath fading can lead to reduced signal quality, increased bit error rates, and decreased system capacity. The received signal is the sum of multiple, delayed, and attenuated components (paths), leading to constructive or destructive interference, time dispersion, and Doppler shifts

In this experiment, amplitude variation is observed with the variation in time corresponding to in phase and quadrature components of the signal. In a Rayleigh fading channel, the received signal amplitude follows a Rayleigh distribution, and the probability density function (PDF) is given by ,

Equations

$$p_r(r) = \frac{r}{\sigma^2} \exp\left(-\frac{r^2}{2\sigma^2}\right); \quad r \geq 0$$

where $2\sigma^2 = E(r^2)$ is the mean power of the signal
 $E(\cdot)$ denotes statistical averaging.

$$R = \sqrt{X^2 + Y^2}$$

where 'r' is the signal amplitude and σ^2 is the variance of the Gaussian random variables representing the in-phase and quadrature components.

There are two channel model-

- **Rayleigh Fading (No LOS):**
 - **Scenario:** Models environments with many reflectors (e.g., urban areas) and no direct path between transmitter and receiver.

- **Distribution:** Amplitude follows a Rayleigh distribution.
- **Characteristics:** Significant fades, often 30-40 dB.

Rician Fading (With LOS):

Scenario: Models scenarios with a strong line-of-sight signal (e.g., satellite links, line-of-sight microwave).

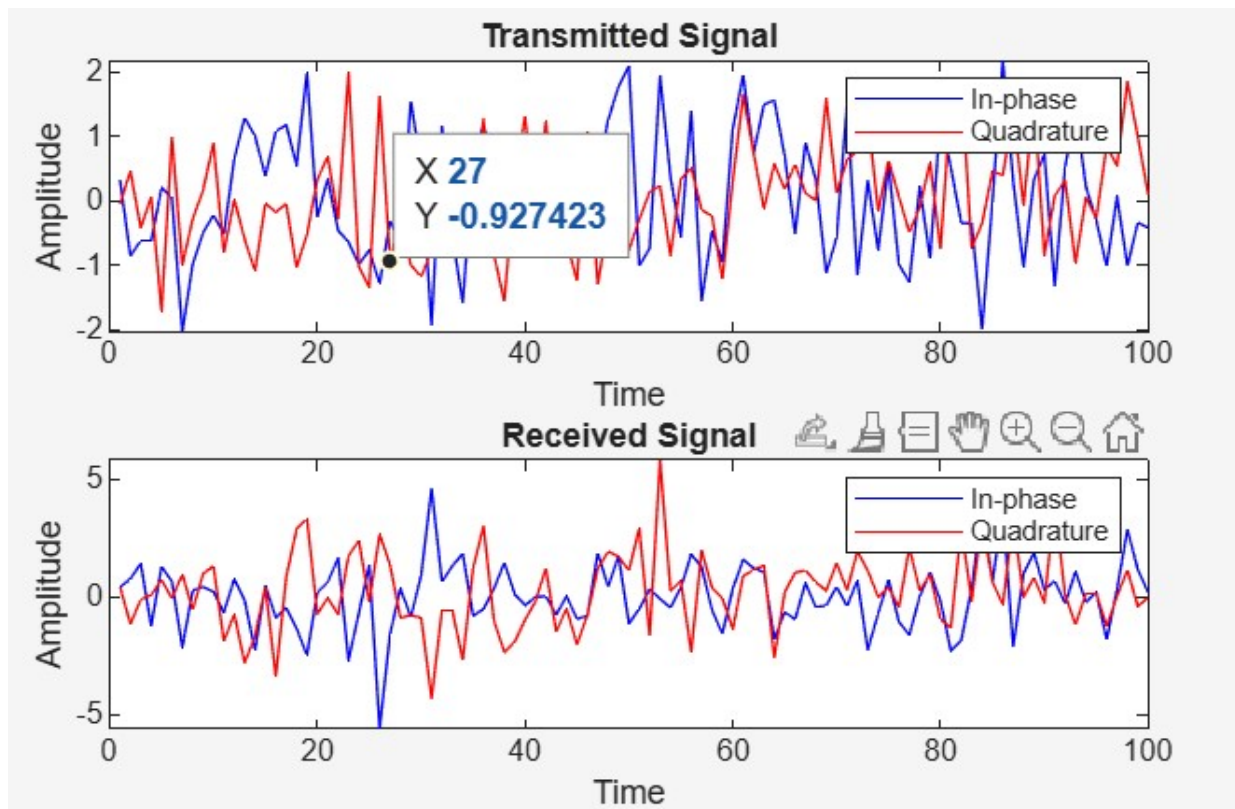
- **Distribution:** Amplitude follows a Rician distribution.
- **Characteristics:** Lower-magnitude variations (typically 10 dB).

Here, we consider Experiment related to Rayleigh Fading Model.

MATLAB code:

```
% Simulation parameters
numSamples = 100; % Number of samples
numPaths = 3;
% Number of multipath paths
fadePower = 0.5; % Fading power
% Generate Rayleigh fading channel coefficients
h = sqrt(fadePower/2)*(randn(numPaths, numSamples) +
1i*randn(numPaths, numSamples));
% Generate transmitted signal
txSignal = randn(1, numSamples) + 1i*randn(1, numSamples);
% Simulate multipath fading channel
rxSignal = zeros(1, numSamples);
for path = 1:numPaths
    rxSignal = rxSignal + h(path, :) .* txSignal;
end
% Plot the transmitted and received signals
t = 1:numSamples;
figure;
subplot(2,1,1);
plot(t, real(txSignal), 'b', t, imag(txSignal), 'r');
title('Transmitted Signal');
legend('In-phase', 'Quadrature');
xlabel('Time');
ylabel('Amplitude');
subplot(2,1,2);
plot(t, real(rxSignal), 'b', t, imag(rxSignal), 'r');
title('Received Signal');
legend('In-phase', 'Quadrature');
xlabel('Time');
```

```
ylabel('Amplitude');
```



Experiment No: 7

Date: _____

Modelling and simulation of Free Space Path Loss***Aim: To determine the free space path loss and the received power for different distances and path loss exponents using MATLAB program.*****Tools Required:** Matlab R2015a/ Simulink library

Theory: The free space path loss, also known as FSPL, is the loss in signal strength that occurs when an electromagnetic wave travels over a line-of-sight path in free space. In these circumstances there are no obstacles that might cause the signal to be reflected refracted, or that might cause additional attenuation. etc. Path loss (PL) refers to the loss or attenuation of a propagating electromagnetic signal (or wave) encounters along its path from transmitter to the receiver. As a result of path loss, the received signal power level is several orders below the transmitted power level. The received power level is dependent on factors such as transmission power, antenna gains, frequency of operation and the distance between the transmitter and the receiver. Like any other gain or attenuation, path loss is also expressed in decibel (dB).

The path loss is expressed mathematically as

$$PL(dB) = 20 \log_{10}(4\pi d/\lambda) \text{---(1)}$$

d is the distance between the transmitter and receiver and λ is the wavelength of the signal.

As said earlier, the received power level depends on the path loss. The expression for the received power at a distance d from the transmitter is expressed mathematically using Friis' transmission equation as

$$P_r = P_t G_t G_r (\lambda/4\pi d)^2 \text{---(2)}$$

By definition, path loss is the ratio of the transmitted power to the received power. From the above equation, we can get the ratio of P_t and P_r .

$$PL(dB) = 10 \log_{10}(P_t/P_r) = -10 \log_{10}[G_t G_r (\lambda/4\pi d)^2] \text{-----(3)}$$

If the antenna gains are assumed to be unity, then eq.3

$$PL(dB) = 20 \log_{10}(4\pi d/\lambda) \text{---(5)}$$

We can observe that PL is a function of square of the signal frequency and the distance between the transmitter and receiver. The received power level depends on the path loss. More the path loss, less is the available received power. We aim to analyse the path loss to maximize the power level available at the receiver.

MATLAB code:

```
function analyze_received_power(ptx, distances, freq,
path_loss_exponents)
    fprintf('Received Power (dBm) for Different Distances and Path
Loss Exponents: \n');
    fprintf('Distance (m) |');
    for exponent = path_loss_exponents
        fprintf(' PL Exponent: %.1f |', exponent);
    end
    fprintf('\n');

    % Dynamic separator length
    separator_length = 15 + length(path_loss_exponents) * 20;
    fprintf(repmat('-', 1, separator_length));
    fprintf('\n');

    % Convert power from Watts to dBm
    ptx_dbm = 10 * log10(ptx * 1000);
    for i = 1:length(distances)
        distance = distances(i);
        fprintf('%-12d |', distance);

        for j = 1:length(path_loss_exponents)
            exponent = path_loss_exponents(j);
            fspl = free_space_loss(distance, freq);

            % Reference distance d0 = 1m
            received_power = ptx_dbm - fspl - 10 * exponent *
log10(distance / 1);

            fprintf('%10.2f dBm |', received_power);
        end
        fprintf('\n');
    end
end

function fspl = free_space_loss(distance, freq)
    % Calculates free space path loss in dB.
```

```

c = 299792458; % Speed of light in meters per second (m/s)
wavelength = c / freq;
fspl = 20 * log10(4 * pi * distance / wavelength);
end

% Example usage
ptx = 0.01; % Transmit power in watts (10 mW)
distances = [10, 50, 100, 200]; % List of distances in meters
freq = 2.4e9; % Frequency in Hz (2.4 GHz)
path_loss_exponents = [2, 3, 4]; % List of path loss exponents (n)

analyze_received_power(ptx, distances, freq, path_loss_exponents);

% Example usage
ptx = 0.01; % Transmit power in watts (10 mW)
distances = [10, 50, 100, 200]; % List of distances in meters
freq = 2.4e9; % Frequency in Hz (2.4 GHz)
path_loss_exponents = [2, 3, 4]; % List of path loss exponents

% Initialize received power array
received_powers = zeros(length(distances),
length(path_loss_exponents));

% Convert power from Watts to dBm
ptx_dbm = 10 * log10(ptx * 1000);

% Compute received power values
for i = 1:length(distances)
    distance = distances(i);
    for j = 1:length(path_loss_exponents)
        exponent = path_loss_exponents(j);
        fspl = free_space_loss(distance, freq);

        % Reference distance d0 = 1m
        received_powers(i, j) = ptx_dbm - fspl - 10 * exponent *
log10(distance / 1);
    end
end

% Plot received power vs. distance
figure;
hold on;

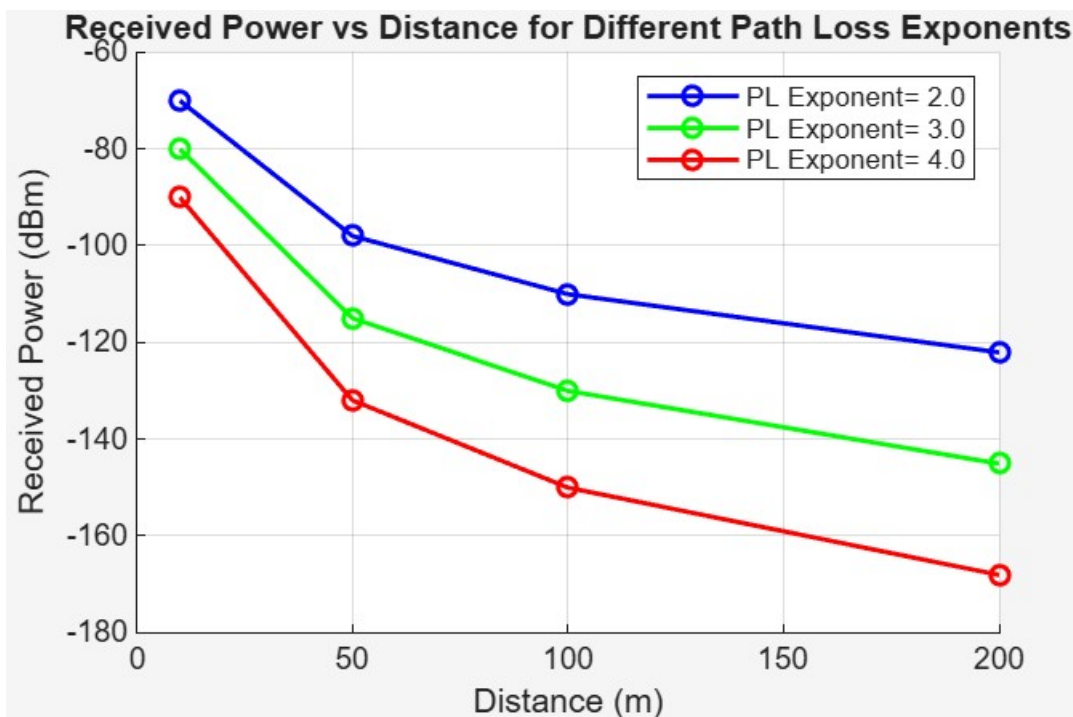
```

```

colors = ['b', 'g', 'r', 'm', 'k']; % Define colors for different exponents
for j = 1:length(path_loss_exponents)
    plot(distances, received_powers(:, j), '-o', 'Color', colors(mod(j-1,
length(colors)) + 1), ...
'DisplayName', sprintf('PL Exponent= %.1f',
path_loss_exponents(j)), 'LineWidth', 1.5);
end
hold off;

xlabel('Distance (m)');
ylabel('Received Power (dBm)');
title('Received Power vs Distance for Different Path Loss Exponents');
legend('show');
grid on;

```



Experiment No: _

Date: _____

Two ray channel, Okumura and Hata model

Aim: To evaluate median path loss for Two- Ray, Okumura model and Hata model for outdoor propagation.

Tools Required: MATLAB R2015a/ Simulink library

Theory: The Okumura model for Urban Areas is a Radio propagation model that was built using data. mura Model. Okumura's model is one of the most widely used models for signal prediction in urban areas. This model is applicable for frequencies in the range 150MHz to 1920 MHz (although it is typically extrapolated up to 3000 MHz) and distances of 1 km to 100 km. It can be used for base station antenna heights ranging from 30 m to 1000 m.

Okumura model was built into three modes: the ones for urban, suburban and open areas. The model for urban areas was built first and used as the base for others. Okumura developed a set of curves giving the median attenuation relative to free space (A_{mu}), in an urban area over a quasi-smooth terrain with a base station effective antenna height (h_{te}) of 200 m and a mobile antenna height (h_{re}) of 3 m.

Coverage

Frequency = 150 MHz to 1920 MHz

Mobile Station Antenna Height: between 1 m and 10 m

Base station Antenna Height: between 30 m and 1000 m

Link distance: between 1 km and 100 km

Hata Model:

The Hata model is an empirical formulation of the graphical path loss data provided by Okumura, and is valid from 150 MHz to 1500 MHz. Hata presented the urban area propagation loss as a standard formula and supplied correction equations for application to other situations. Hata Model predicts the total path loss along a link of terrestrial microwave or other type of cellular communications. This particular version of the Hata model is applicable to the radio propagation within urban areas. This model is suited for both point-to-point and broadcast transmissions and it is based on extensive empirical measurements taken.

Hata Model for Urban Areas is formulated as:

$$L_U = 69.55 + 26.16 \log f - 13.82 \log h_B - CH + [44.9 - 6.55 \log h_B] \log d.$$

For small or medium sized city,

$$CH = 0.8 + (1.1 \log f - 0.7) h_M - 1.56 \log f.$$

and for large cities,

$$CH = 8.29 (\log (1.54 h_M))^2 - 1.1, \text{ if } 150 \leq f \leq 200$$

$$CH = 3.2 (\log (11.75 h_M))^2 - 4.97, \text{ if } 200 \leq f \leq 1500$$

Where,

L_U = Path loss in Urban Areas (dB)

h_B = Height of base station Antenna. (m)

h_M = Height of mobile station Antenna. (m)

f= Frequency of Transmission (MHz).

CH = Antenna height correction factor

d= Distance between the base and mobile stations (km).

The term "small city" means a city where the mobile antenna height not more than 10 meters. i.e.

$1 \leq h_m \leq 10\text{m}$

Two ray Ground reflection model-

$$PL(\text{dB}) = 40\log d - (10\log G_t + 10\log G_r + 20\log h_t + 20\log h_r)$$

$$P_r = P_t G_t G_r \frac{h_t^2 h_r^2}{d^4}$$

MATLAB Program:

```
clc;
```

```
clear;
```

```
close all;
```

```
%Enter Input Parameters%
```

```
f = input('Enter frequency (MHz): '); % 150–1500 MHz
```

```
hb = input('Enter base station height (m): '); % 30–200 m
```

```
hm = input('Enter mobile height (m): '); % 1–10 m
```

```
% Distance range (km)
```

```
d = 1:0.5:20;
```

```
% Convert frequency to Hz for two-ray model
```

```
f_hz = f * 1e6;
```

```
c = 3e8;
```

```
lambda = c / f_hz;
```

```
if f < 150 || f > 1500
```

```
    error('Frequency out of valid range (150–1500 MHz)');
```

```
end
```

```
if hb < 30 || hb > 200
```

```
    error('Base station height out of range (30–200 m)');
```

```
end
```

```

if hm < 1 || hm > 10
    error('Mobile height out of range (1-10 m)');
end

%Initilize path loss for all three models
PL_two_ray = zeros(size(d));
PL_okumura = zeros(size(d));
PL_hata = zeros(size(d));
for i = 1:length(d)
    dist_km = d(i);
    dist_m = dist_km * 1000;

    %Ground Two-Ray Model
    if dist_m < 10*sqrt(hb*hm)
        % Not strictly valid, still computed
        warning('Distance may not satisfy two-ray condition');
    end

    Pr_Pt = (hb^2 * hm^2) / (dist_m^4);
    PL_two_ray(i) = -10*log10(Pr_Pt); % Path loss in dB

    % Okumura Model
    Lf = 32.45 + 20*log10(f) + 20*log10(dist_km);
    Amu = 35 + 20*log10(dist_km); % Approximation
    Ghb = 20*log10(hb/200);
    Ghm = 10*log10(hm/3);
    Garea = 0; % Urban

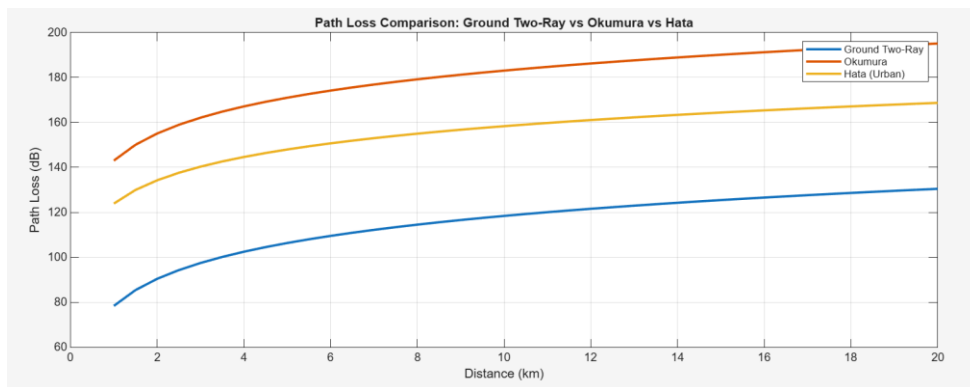
    PL_okumura(i) = Lf + Amu - Ghb - Ghm - Garea;

    % Hata Model (Urban)
    ch = 0.8 + (1.1*log10(f)-0.7)*hm - 1.56*log10(f);

    PL_hata(i) = 69.55 + 26.16*log10(f) ...
        - 13.82*log10(hb) - ch ...
        + (44.9 - 6.55*log10(hb))*log10(dist_km);
end

```

```
end  
  
figure;  
  
plot(d, PL_two_ray, 'LineWidth', 2); hold on;  
  
plot(d, PL_okumura, 'LineWidth', 2);  
  
plot(d, PL_hata, 'LineWidth', 2);  
  
grid on;  
  
xlabel('Distance (km)');  
  
ylabel('Path Loss (dB)');  
  
title('Path Loss Comparison: Ground Two-Ray vs Okumura vs Hata');  
  
legend('Ground Two-Ray', 'Okumura', 'Hata (Urban)');
```



Experiment No:

Date: _____

Analyze the beamforming (Radiation Pattern) parameters of linear array antenna in wireless communication

Tools Required: MATLAB R2015a/ Simulink library

Theory: Beamforming is a signal processing technique that enhances the performance of wireless communication systems. It involves directing the transmission or reception of radio waves in a specific direction, rather than broadcasting signals in all directions equally. This targeted approach allows for improved signal quality, increased range, and reduced interference.

In beamforming, multiple antennas are used to transmit or receive signals. By manipulating the phase and amplitude of each antenna's signal, the system can create constructive interference in the desired direction and destructive interference in other directions. This results in a focused beam of energy that can be steered towards a specific user or location.

Beamforming technology has been widely adopted in various applications such as wireless communication systems (e.g., Wi-Fi routers), radar systems, and cellular networks. It offers several advantages including increased data rates, improved coverage, reduced power consumption, and enhanced overall system capacity.

Benefits of Beamforming

As 5G networks continue their global rollout, enabling smartphones, connected devices, and wide-area networking applications, beamforming emerges as a critical core technology. The high-frequency millimeter wave (mmWave) spectrum used in 5G communications is susceptible to disruptions from obstacles like walls and other barriers, posing challenges for reliable connectivity.

Beamforming plays a crucial role in mitigating these challenges by allowing transmitters to focus their signals in specific directions towards mobile devices, vehicles, or Internet of Things (IoT) devices. This directional transmission not only enhances signal strength but also improves the overall reliability of 5G connections.

Moreover, beamforming will be a key enabler for massive MIMO (Multiple-Input Multiple-Output) technology in 5G networks. Massive MIMO employs a large number of antennas at the base station, which, in conjunction with beamforming, can direct beams both horizontally and vertically towards user devices. This precise beam steering capability not only maximizes throughput but also optimizes spectral efficiency, leading to improved network performance and capacity.

```
clc;
clear;
close all;
```

```
% Parameters
frequency = 3e9; % Frequency (3 GHz)
lambda = 3e8 / frequency; % Wavelength
numElements = 8; % Number of elements in the array
d = lambda / 2; % Element spacing (in wavelengths)
theta_steering = 30; % Desired steering angle (degrees)
```

```

% Angle range for radiation pattern
angles = linspace(-90, 90, 180); % Angles from -90 to 90 degrees

% Define the element positions in a linear array
elementPositions = (0:numElements-1) * d; % Positions of the
elements

% Calculate the array factor
AF = zeros(size(angles)); % Initialize array factor

for m = 1:numElements
    beta = exp(1j * (2 * pi / lambda) * (elementPositions(m) *
sind(theta_steering))); % Phase shift for steering
    AF = AF + beta * exp(1j * (2 * pi / lambda) *
(elementPositions(m) * sind(angles))); % Array factor
    calculation
end

% Normalize the array factor
AF = abs(AF);
AF = AF / max(AF); % Normalize to maximum value

% Plot the radiation pattern
figure;
polarplot(deg2rad(angles), AF, 'LineWidth',2);
title('Beamforming Pattern of Patch Antenna Array');
ax = gca;
ax.Theta = -90:30:90; % Set theta
grid on;

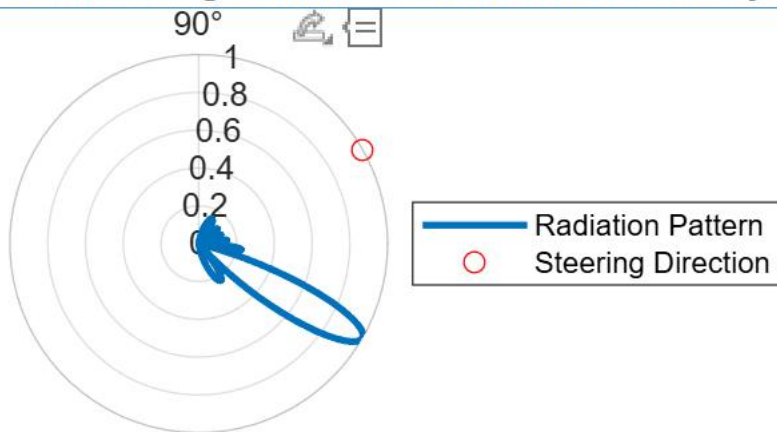
% Display the main lobe direction
hold on;
polarplot(deg2rad(theta_steering), max(AF), 'ro'); % Marking the
steering direction
legend('Radiation Pattern', 'Steering Direction');
hold off;

```

Observations:

Change the parameters **Desired steering angle** and observe the steering direction and main beam

Beamforming Pattern of Patch Antenna Array



Beamforming Pattern of Patch Antenna Array

